NLP UNIT-II

- **Subject:** Artificial Intelligence / NLP
  Presented by: Duggirala SriRamyaKrishna
  Department: CSE
- College:- Sir C R Reddy Engineering College
- Designation:- Asst professor
- Mail:-ramya999mr@gmail.com

- N-grams, Evaluation, Smoothing, Interpolation, Backoff
- Word Classes, POS Tagging
- Rule-based, Stochastic, Transformation-based tagging
- Hidden Markov Models, Maximum Entropy Models

What is Word Level Analysis?

Word level analysis means:

- Studying language **word by word**
- Understanding how words come together
- Predicting the **next word** in a sentence

What is an N-gram Model?

An N-gram model:

- Predicts the **next word**
- Uses previous words
- Learns from past text (training data)

Example:

- If you type:
- “How are”
- The model predicts:
- “How are you”
- **This method is based on probability.**

Duggirala Sri Ramya Krishna

What are Unsmoothed N-grams?

- An **unsmoothed N-gram model** calculates probabilities using **only the text it has seen during training**.
- It uses a method called:

Maximum Likelihood Estimation (MLE)

- The idea is simple:
- If something happened many times earlier, it will probably happen again.
  If it never happened before, it is considered impossible.

Problems of unsmoothed N grams

Sparse Data Problem:

That means:

- Most word combinations **never appear**
- Even large corpora miss many sentences
- Language is creative
- New word combinations happen daily

Sparse data means your dataset does not contain enough examples for most word combinations, leaving many patterns unseen or rare.

Evaluating N-Gram Models

What is Evaluation?

- Evaluation means checking how well an N-gram model predicts real language.

Why evaluate?

- We evaluate N-gram models to:
- Choose the best model
- Improve prediction accuracy
- Avoid using poor models

Duggirala Sri Ramya Krishna

Train–Test Method

Data is split into:

- **Training set** → Build the model
- **Test set** → Evaluate model

Rule:

Never test on training data (it produces false results)

Evaluation Metric: Perplexity

What is Perplexity?

- Perplexity measures How confused the model is while predicting text.
- Lower perplexity = Better model
  Higher perplexity = Worse model

Formula:

$$\text{perplexity} = \sqrt[N]{1/p(\text{sentence})}$$

Where:

P = probability of the sentence

N = number of words

Model	Perplexity
Unigram	High
Bigram	Lower
Trigram	Lowest

Conclusion:

More context = better prediction

Problem with Unsmoothed N-grams:

- Unseen words get probability = 0
- Sentence probability becomes zero
- Perplexity becomes infinite

Solution:

- ✓ Smoothing techniques
  (Gives small probability to unseen words)

What is Smoothing?

Meaning:

- Smoothing is a technique used to:
- Give small probabilities to unseen word sequences.

Why is it needed?

- In unsmoothed N-grams:
- Unseen words get probability = 0
- Full sentence probability becomes 0
- Evaluation fails

.

Why Smoothing is Important

- Without smoothing:
  - ✗ New sentences fail
  - ✗ Model rejects valid language
  - ✗ Perplexity becomes infinite
  - ✗ System becomes unreliable

- With smoothing:
  - ✓ All words get some probability
  - ✓ New sentences are accepted
  - ✓ Evaluation becomes possible
  - ✓ Model performs better

Example with and without Smoothing

Training Data:

- "I love tea"
- Bigram: love tea = 1
  Bigram: love milk = 0
- **Without Smoothing:**
- P(milk|love)=0
- **With Smoothing:**
- P(milk|love)>0

Conclusion:

Smoothing saves the model.

How Smoothing Works (Simple Idea)

Core idea:

- Take some probability from frequent words and share it with unseen words.
- Like sharing money:
  A rich person gives ₹1
  A poor person gets ₹1
- Result:
  Everyone has at least something.

Types of Smoothing (From Textbook)

Method	Meaning
Laplace(add one)	Add 1 to every count
Lidstone	Add small value
Good-Turing	Uses rare word counts
Kneser-Ney	Uses context awareness

Problem Statement

In N-gram models:

- Many sentences never appear in training data
- New words always come
- Predictions fail
- Main problem:
- Zero probability for unseen words

Duggirala Sri Ramya Krishna

1. Interpolation (mixing multiple N-gram models)

The idea is:

- "Don't put all trust in one model; combine them with weights."
- Interpolation means:
- Mixing different N-gram models.
- Instead of using:
- Only trigram
- Only bigram
- Only unigram
- We use:
  ✓ All of them together

Duggirala Sri Ramya Krishna

How Interpolation Works

We take probability from:

- Trigram
- Bigram
- Unigram And combine:
- Final Probability=λ1(Trigram)+λ2(Bigram)+λ3(Unigram)
- Where: λ values add to 1
- They decide importance

Interpolation Example

Model	Probability
Trigram	0.5
Bigram	0.3
Unigram	0.2

With:
$\lambda_1 = 0.5, \lambda_2 = 0.3, \lambda_3 = 0.2$
Final probability=0.5(0.5)+0.3(0.3)+0.2(0.2)
=0.25 + 0.09 + 0.04
= 0.38

Backoff

- Backoff uses the idea:
- "First try the **most specific** model;
  if there is no evidence, then **back off** to a simpler one."

Meaning:

- Backoff means:
- If detailed model fails, use simpler one.

Order:

- Trigram
- Bigram
- Unigram
- Use whichever is available.

Backoff Example

Sentence:

- "I love milk"
- Trigram:
  "I love milk" → ✗ Not found
- Try bigram:
  "love milk" → ✓ Found
- Use bigram probability.
- If not found:
  Use unigram.

Interpolation vs Backoff

Interpolation	Backoff
Use all models	Use one model
Always combine	Use only when needed
More accurate	Simple
Needs tuning	Fast

Duggirala Sri Ramya Krishna

Word Class / Class-based N-grams

- Instead of modeling words directly:
- we Group words into classes / clusters:
  - CITYNAME: London, Beijing, Shanghai, Delhi
  - AIRLINE: AirIndia, Indigo, Emirates
  - FOOD: tea, coffee, milk

- Build N-gram model over classes (like CITYNAME, AIRLINE,… instead of the actual names).

Word-Class Model – Idea

- Word-class model:
- Replace words with categories (classes).
- Instead of:
  "I like milk", "I like coffee"
- Use:
  "I like <FOOD>"

Word-Class Model – Examples

Class	Words
CITY	Delhi, London
FOOD	tea, milk
PERSON	teacher

Word-Class N-gram Model: Numerical Example

Training Sentences:

- I like tea
- I like coffee
- I like milk
- You like tea

Word-Class Mapping:

Word	Class
I, You	PERSON
like	VERB
tea, coffee, milk	FOOD

Converted Class Sentences:

- All become:
- PERSON → VERB → FOOD
- ✓ The model learns relationships between classes instead of words.

How Word-Class Model Works

Probability is computed by:

- Predicting the class
- Predicting word within class
- So:

$$P(word|previous)=P(Class|previousClass) \times P(word|Class)$$

Class Probability & Word Probability

1. Class Probability Formula:

- $P(\text{Class}[i] \mid \text{Class}[i-1]) = \dfrac{\text{Count}(\text{Class}[i-1], \text{Class}[i])}{\text{Count}(\text{Class}[i-1])}$

- Numerator → how often class pair occurs
- Denominator → how often first class occurs

2. Apply Formula:

- From training data:
- "VERB → FOOD" count = 4
- "VERB" count = 4
- P(FOOD|VERB)=4/4=1

✓ After VERB, FOOD always appears in training data.

3. Word Probability Formula:

- $P(\text{word}|\text{class}) = \dfrac{\text{Count(word)}}{\text{Count(class)}}$

FOOD Class Counts:

Word	Count
tea	2
coffee	1
milk	1

Total FOOD count = 4

- **Calculate**
- P(tea|FOOD)=2/4=0.5
- P(milk|FOOD)=1/4=0.25
- P(coffee|FOOD)=1/4=0.25

Final Probability

- Final Formula:
- P(word | previous word) = P(class | previous class) × P(word | class)

Example 1: Find P(tea | like)

- "like" → VERB
- "tea" → FOOD

P(tea | like)= P(FOOD|VERB) × P(tea | FOOD)

= 1 × 0.5

=0.5

- P(milk | like)= 1×0.2.5 = 0.25
- P(coffee | like)= 1×0.25 = 0.25

- ✓ Final Result Table:

Word	Final Probability
tea	0.5
milk	0.25
coffee	0.25

What is POS Tagging?

POS tagging means:
Identifying what role each word plays in a sentence.
Roles such as:

- Noun
- Verb
- Adjective
- Adverb
- Preposition, etc.

Example:
I like apples

- Tagged : I/PRON like/VERB apples/NOUN

What is a Tagset?

- A **Tagset** is : A list of labels used by the computer to tag words
- Common Tag Examples:

Tag	Meaning	Example
NOUN	naming word	book
VERB	action word	run
ADJ	describing word	good
ADV	describing action	fast
PRON	pronoun	he
PREP	preposition	in
DET	article	the

POS Ambiguity (Main Challenge)

- One word → many meanings.

Word	Tag
book	NOUN
book	VERB
can	NOUN
can	VERB

- **Problem** : Computer must choose the **correct tag using the sentence**.

1. Rule-Based POS Tagging

Uses grammar rules written manually.

- Rules are based on:
- Word position
- Previous word
- Word endings

Example Rules:

- Rule 1:
  If a word follows “to”, tag it as VERB
- to play → play/VERB
- Rule 2:
  If word ends with "-ly", tag it as ADVERB
- quickly/ADV

2. Statistical (HMM) POS Tagging

Uses **probability** instead of rules.

- It calculates:
- Which tag usually comes next
- Which word belongs to which tag

Example:

The dog barks

- From training:
- DET → NOUN is common
- NOUN → VERB is common
- So:
  The/DET dog/NOUN barks/VERB

3. Transformation-Based Tagging (Brill Tagger)

Starts with simple tagging

Then **corrects mistakes using learned rules.**

Example:

- Initial tagging:
- Book/NN a/DT ticket/NN
- Rule learned:
  Change NN → VB if word is first and followed by DT
- Final:
- Book/VB a/DT ticket/NN

4. Machine Learning-Based Tagging

Uses trained models to learn deep patterns.

- Examples:
- Maximum Entropy
- CRF
- Neural Networks
- **Example:**
- From thousands of sentences, model learns:
  "runs" is usually VERB
  "beautiful" is ADJ
- She runs fast
  She/PRON runs/VERB fast/ADV

Comparison of POS Tagging Methods

Method	Uses	Strength
Rule-Based	Grammar rules	Easy to understand
HMM	Probability	Good accuracy
Brill	Error fixing	Learns rules
ML Based	Training data	Best for large data

Duggirala Sri Ramya Krishna

✓ RULE-BASED POS TAGGING (3 EXAMPLES)

Sentence	Rule Used	POS Result
I want to play	Word after "to" → VERB	play = VERB
She talks loudly	Word ending with "-ly" → ADVERB	loudly = ADVERB
He bought a car	Word after "a" → NOUN	car = NOUN
They are happy	Word after "are" → ADJ	happy = ADJ
Rahul is running	Word after "is" → VERB	running = VERB

✓ STOCHASTIC (STATISTICAL) TAGGING (5 EXAMPLES)

Sentence	Tag Options	Final Tag
Time flies	N-V vs V-N	Time = NOUN, flies = VERB
Book flight	N-N vs V-N	Book = VERB, flight = NOUN
Can you can	AUX-V vs AUX-N	Last "can" = VERB
Light work	A-N vs N-V	Light = ADJ, work = NOUN
Race cars	N-N vs V-N	Race = NOUN, cars = NOUN

✓ TRANSFORMATION-BASED TAGGING (5 EXAMPLES)

Initial Tagging	Rule Applied	Final Correct Tag
Book/NN a/DT ticket	NN → VB if next is DT	Book = VERB
The/DT race/VB car	VB → NN if prev is DT	race = NOUN
fast/NN car	NN → JJ if next is NN	fast = ADJ
I/PRON like/NOUN tea	NOUN → VERB if prev is PRON	like = VERB
She/PRON will/NOUN go	NOUN → AUX if next is VERB	will = AUX

Hidden Markov Model (HMM)

Hidden Markov Model (HMM) is a **statistical model** used for:

- POS tagging
- Speech recognition
- Sequence labelling

It assumes:

- Tags are hidden
- Words are visible
- Tag depends on previous tag (Markov assumption)

HMM Components

Parts of HMM:

- States → POS tags (NOUN, VERB, etc.)
- Observations → Words

- Transition Probability : What tag comes after what tag?
  $P(Tag_i \mid Tag_{i-1})$
  Ex:P(VERB | NOUN)
- Emission Probability : Which word comes from which tag?
  P(Word | Tag)
  Example:
  P("dog" | NOUN)

HMM Working Example

- Sentence to tag:
- Cats drink
- Goal:
  Find correct POS tags for:
- Cats → ?
  drink → ?
- Possible Tags:
- NOUN (N)
- VERB (V)

Training Data

- **From where probabilities come?**
- We use training sentences:
- Dogs eat
- Cats eat
- Dogs bark
- Boys play
- Cats drink
- All sentences follow pattern:
- NOUN → VERB

Emission Probabilities Table

- **Word given Tag Probabilities**
- Calculated from frequency in training data.
- P(word|tag)=Count(word as tag) /Total(tag)

Counts from Training Data:

Word	Tag	Count
Cats	NOUN	2
drink	VERB	1
Cats	VERB	0
drink	NOUN	0

Total Tags Count:

Tag	Total
NOUN	5
VERB	5

Emission Probabilities:

Word	NOUN	VERB
Cats	2/5 = 0.4	0
drink	0	1/5 = 0.2

Transition Probabilities Table

Tag-to-tag= $p(tag_i \mid tag_{i-1}) = Count(tag_{i-1}, tag_i) / Count(tag_{i-1})$

Counts:

From → To	Count
START → NOUN	5
NOUN → VERB	5
VERB → END	5

Transition Probabilities:

Transition	Probability
START → NOUN	5/5 = 1
NOUN → VERB	5/5 = 1

Try all Possible Tag Sequences

- We compute probability for each tag sequence.

Case 1: Cats / NOUN, drink / VERB

P = P(N | START) × P(Cats | N) × P(V | N) × P(drink | V)

= 1 × 0.4 × 1 × 0.2

= 0.08

Case 2: Cats / VERB, drink / NOUN

=P(V|START) × P(Cats|V) × P(N|V) × P(drink|N)

=0 × .....

=0

What did we learn?

- Best Sequence (Highest Probability)

Sequence	Probability
Cats/N drink/V	**0.08** ✓
Cats/V drink/N	0 ✗

Final Tagging:

- Cats / NOUN
  drink / VERB

What is Maximum Entropy?

Maximum Entropy is a **machine learning model** used to predict labels like:

- POS tags
- Named entities
- Sentiment classes

Simple Idea:

- Do not assume anything unless the data shows it.
- MaxEnt selects the model that:
- Matches the training data
- Has the **least bias**
- Is most general

How MaxEnt Works

MaxEnt makes decisions using **features**, not rules.

Example Features:

- Current word
- Previous word
- Next word
- Capital letter?
- Ending of word (-ing, -ed)
- Word length
- Position in sentence

Example (POS Tagging)

Sentence : Dogs fish

- Word: **fish**

Features extracted:

- Previous word = dogs
- Word is "fish"
- Ends in "h"
- No capital letter

Model sees in training:

- "dogs fish" → VERB mostly
- "a fish" → NOUN mostly

Because previous word is "dogs":

✓ fish is predicted as **VERB**

- **Sentence:** She will book tickets
- Let's predict the tag for the word **"book"**
- Features observed:

Feature	Value
Previous word	will
Next word	tickets
Word shape	small letters
Verb ending	no

MaxEnt has learned:

- "will + verb" is very common
- "book" after "will" is usually a verb

Decision:

- book → **VERB**
- Tagged sentence:
  She/PRON will/AUX book/VERB tickets/NOUN


Thank
you!!
...